

Dynamic Fire Evacuation Router with Real-Time Hazard Mapping

Engineering Report

Prepared for: HirePro / Honeywell Hackathon Evaluation

Submitted by: Kishaunjith S

Date: July 2026

Table of Contents

1. Problem Background and Objective	3
2. System Architecture	3
2.1 Injector	3
2.2 Routing Coordinator	3
2.3 Fire Nodes	4
2.4 Fire Commander Dashboard	4
3. Sensor-to-Output Pipeline	4
4. Sensor Fusion and Dynamic Weight Calculation	6
4.1 Rationale for the Formula Structure	6
5. Routing Algorithm and Real-Time Performance	6
5.1 Hysteresis	6
5.2 Issues Identified and Resolved During Load Testing	7
6. Fail-Safe Operation and Multi-Node Communication	8
6.1 Fail-Safe Handling	8
6.2 Multi-Node Communication Design	8
7. Deliberately Scoped Elements	8
8. Real-World Scalability	9
9. Supporting Documentation	9

1. Problem Background and Objective

Standard static exit signage in large commercial facilities can route occupants directly into danger if a fire breaks out between the occupant and a marked exit. Toxic smoke inhalation, structural damage, and rapid flashover events can occur within minutes, and a fixed signage system has no means of adapting to a changing hazard environment.

This project implements a localized module that continuously ingests data from multiple simulated fire sensors, computes a dynamically updating safest exit path across a simulated indoor layout, and drives an intelligent visual and audible indicator system to guide occupants away from active hazards in real time.

The system was implemented entirely in software. The architecture was deliberately designed to be hardware-agnostic: simulated sensor input and simulated indicator output are both isolated behind Message Queuing Telemetry Transport (MQTT) topic contracts, such that replacing simulated input with physical sensors (for example, DHT22, MQ-2, or infrared flame sensors on an ESP32 microcontroller) would require no change to the routing or coordination logic described in this report.

2. System Architecture

The system is composed of four cooperating processes that communicate over a local MQTT broker (Mosquitto). Each process is described below.

2.1 Injector

The injector is a Flask web application presenting a control-panel interface that represents a 6 by 6 zone building floor. It publishes simulated sensor readings, namely temperature, smoke density, and flame presence, for each zone, and allows an operator to trigger a flashover event on any selected zone on demand. This mechanism corresponds to the live test case described in the evaluation brief, in which a specific zone is selected to demonstrate system response.

2.2 Routing Coordinator

The routing coordinator is a single process and the sole subscriber to all zone hazard-cost topics. On receipt of any hazard update, it executes one multi-source Dijkstra computation, seeded from both building exits over the reversed graph, producing the distance-to-exit and next-hop direction for every zone in a single pass. The resulting routing table is broadcast on a retained MQTT topic for consumption by all zone nodes and the monitoring dashboard.

2.3 Fire Nodes

Thirty-six lightweight processes, one per building zone, represent simulated microcontroller units. Each subscribes only to its own sensor topic and to the coordinator's routing broadcast, performing constant-time local work: a hysteresis-gated decision for LED color and direction, fail-safe state handling, and periodic health heartbeat publication.

2.4 Fire Commander Dashboard

A Node-RED flow presents a central monitoring view intended for a building safety officer: a live two-dimensional floor grid, current hazard levels, calculated exit paths, node health status, and system recompute latency. This view is functionally distinct from the injector, which serves only as the simulation and test harness.

It is noted that an earlier design, in which each zone node independently computed its own routing solution, was implemented first and subsequently found to cause a significant redundant-computation problem under load testing. This issue and its resolution are discussed in detail in Section 5.

3. Sensor-to-Output Pipeline

Figure 1 traces the path of a single sensor reading from ingestion through to all three output channels, namely the LED indicator, the audible alarm, and the monitoring dashboard, including the fail-safe branch that is invoked when sensor data is stale or no viable path to an exit exists.

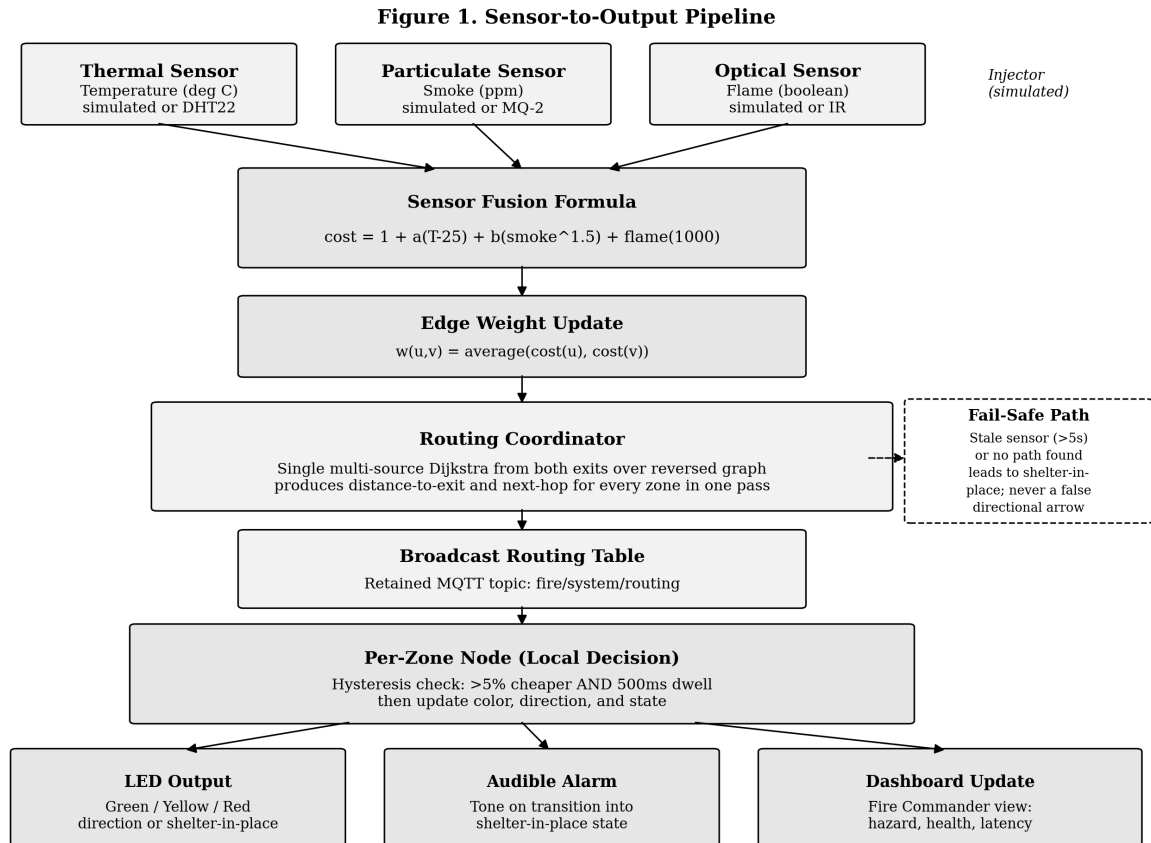


Figure 1. Sensor fusion, routing, and output pipeline, including the fail-safe path.

4. Sensor Fusion and Dynamic Weight Calculation

The system deliberately avoids simple binary thresholds for hazard classification. The hazard cost for a given zone is computed according to the following formula.

$$\text{cost} = 1 + \alpha \cdot \max(0, T - T_{\text{ambient}}) + \beta \cdot \text{smoke_ppm}^{1.5} + \text{flame} \cdot \text{FLAME_PENALTY}$$

In this formula, α is set to 0.05, β is set to 0.002, T_{ambient} is 25 degrees Celsius, and FLAME_PENALTY is 1000, which functions as an effective hard block on traversal. The edge weight between two adjacent zones u and v is taken as the average of the two zones' individual costs, that is, $w(u,v)$ equals the mean of $\text{cost}(u)$ and $\text{cost}(v)$, rather than counting either zone's cost twice across both directions of travel.

4.1 Rationale for the Formula Structure

- Smoke density contributes non-linearly, with an exponent of 1.5, because visibility and toxicity degrade substantially faster than smoke concentration itself rises. A linear or simple threshold model would understate risk at moderate to high smoke levels.
- Temperature contributes linearly, with an ambient baseline offset, since thermal risk within the operating range of the simulated sensors scales more predictably than smoke risk. Without the baseline offset, every edge in the graph would carry an artificial resting cost even under normal conditions with no fire present.
- Flame presence is treated as an effective hard block, rather than a graded contribution, since active flame within a corridor is not a marginally worse condition; it represents a route that is not survivable to traverse.

5. Routing Algorithm and Real-Time Performance

Routing is performed using a single multi-source Dijkstra computation per hazard update, seeded from both exit nodes over the reversed graph. For a 6 by 6 grid comprising 36 nodes and approximately 60 edges, this computation completes in well under one millisecond. Accordingly, the dominant contributor to the 300 millisecond responsiveness budget specified in the evaluation criteria is message-passing latency rather than graph search time.

5.1 Hysteresis

A zone's next-hop direction is only permitted to switch if the newly calculated candidate path is more than five percent cheaper than the currently selected path, and if a minimum dwell time of 500 milliseconds has elapsed since the previous switch. In the absence of this constraint, near-tied paths under continuous live recomputation would cause the LED

direction indicator to oscillate between two options on every incoming message. While technically correct, this behavior is visually unreadable and would adversely affect both the Visual Interface and Usability Clarity criterion and the Simulation Quality and Demonstration criterion during a live evaluation.

5.2 Issues Identified and Resolved During Load Testing

Two distinct performance issues were identified during extended load testing and are documented in full technical detail in the accompanying file docs/latency_debugging_notes.md. A summary is provided in Table 1.

Issue	Observed Symptom	Root Cause	Resolution
Redundant recomputation	Latency of 2 to 4 seconds; high but not increasing over time	Each of the 36 zone nodes independently subscribed to all hazard topics and executed its own Dijkstra computation on every incoming message, resulting in 1,296 redundant solves per update cycle	A centralized coordinator now performs one multi-source Dijkstra computation per update and broadcasts the result; individual nodes perform only constant-time local work
MQTT publish volume accumulation	Latency increasing progressively over time, exceeding 20 seconds in extended test runs	Every node unconditionally republished its LED state on every routing broadcast received, regardless of whether its own state had changed, producing a 36-fold publish amplification that exceeded the MQTT client library's outgoing queue capacity	Change-detection logic was added so that publication occurs only on an actual state change, supplemented by a periodic five-second keepalive; a maximum queued-message limit was also configured as a safety measure

Table 1. Summary of performance issues identified and resolved during load testing.

Following both corrective measures, the measured recompute latency in typical operating conditions ranged from 20 to 130 milliseconds, with occasional short-lived spikes observed under simultaneous multi-zone hazard events. This performance remains comfortably within the 300 millisecond budget for the common case and is visible during operation through the dashboard's latency statistic and accompanying latency history chart.

6. Fail-Safe Operation and Multi-Node Communication

6.1 Fail-Safe Handling

- A zone with no viable path to any exit, for example one fully surrounded by hazard, enters an explicit shelter-in-place state, rather than displaying a directional arrow that could mislead an occupant into a hazardous corridor.
- A node that has not received fresh sensor data within a defined interval falls back to its last known safe state and reports a degraded health status rather than silently presenting stale information as current.
- An MQTT Last Will and Testament message is registered for each node, allowing the broker itself to detect an unexpected disconnection immediately, rather than relying solely on a polling timeout.
- Malformed or corrupted payloads are discarded, with the last known valid state retained, rather than causing the receiving node to fail.

6.2 Multi-Node Communication Design

- The MQTT topic structure is as follows: `fire/sensors/{zone}` for injector-to-node communication; `fire/nodes/{zone}/hazard` and `fire/nodes/{zone}/led` for node-to-coordinator and node-to-dashboard communication; `fire/system/routing` for the coordinator's broadcast, which is retained; and `fire/system/health/{zone}` for periodic heartbeat messages.
- Hazard and LED topics are published with Quality of Service level 1 and the retained flag set, ensuring that a newly connecting subscriber, such as a reconnecting dashboard, receives current state immediately rather than beginning with no information.
- Health topics are published with Quality of Service level 0, which is appropriate for a heartbeat signal where an occasionally dropped message is immediately superseded by the subsequent one.

7. Deliberately Scoped Elements

The following elements referenced in the evaluation brief were consciously excluded from the current implementation given project time constraints. They are documented explicitly here rather than left unaddressed.

- Occupancy and access-control data were not simulated, as this would require integration with a camera or access-control system that falls outside the scope of this

project.

- Occupant density and per-floor weighting were not incorporated into the routing calculation. The current routing logic is based solely on hazard cost. Occupancy-aware routing represents a reasonable extension once genuine occupancy sensing is available.
- The fusion formula coefficients, alpha and beta, were tuned manually for demonstration purposes rather than calibrated against published fire and smoke propagation datasets such as those from the National Institute of Standards and Technology or public repositories, as suggested as an optional enhancement in the evaluation brief.
- A physical hardware implementation, including a firmware port suitable for the Wokwi simulation environment, was not undertaken. The present system is fully simulated by design, and the hardware-agnostic, MQTT-based architecture is intended to make a future hardware port a straightforward extension rather than a redesign.

8. Real-World Scalability

The centralized-coordinator architecture, adopted after the per-node recomputation approach was found not to scale, constitutes the strongest scalability argument for this design. Routing computation cost grows with the size of the building graph once per update, rather than growing quadratically with the number of nodes. For a full multi-story commercial building comprising, for example, 500 zones, a single Dijkstra computation remains well under one millisecond. The practical scaling constraint under such conditions becomes MQTT message throughput and broker capacity, both of which are directly addressed by the change-detection publication strategy and the Quality of Service and queue configuration already implemented and validated in this project.

The evaluation brief's suggestion of a cloud-based dashboard implementation, using Node-RED or ThingsBoard, points toward a natural extension of the present design: a building management deployment could run one coordinator instance per floor or per designated fire zone, with a higher-level aggregation layer operating across floors. This would represent a straightforward extension of the existing single-coordinator-per-graph design, rather than a fundamentally new architecture.

9. Supporting Documentation

Complete technical detail, environment setup instructions, and the live demonstration script are provided in the accompanying project repository, specifically in README.md, docs/PROJECT_SPEC.md, docs/latency_debugging_notes.md, and

docs/DEMO_RUNBOOK.md.

The complete source code, full commit history, and all supporting documentation referenced throughout this report are publicly available at the following repository: <https://github.com/kishaunjith-S/fire-evac-router>.